nDCG, Worked Out by Hand

The ranking metric that rewards putting the *most* relevant result first — it grades results on a scale and discounts each by its position, then normalises against the perfect ordering. Computed in full on a six-result list.

What this document does. It builds Discounted Cumulative Gain from two ideas — graded relevance and a logarithmic position discount — computes DCG on a ranked list, computes the Ideal DCG by sorting that same list, and divides to get nDCG $\in [0, 1]$. It explains why the metric needs *graded* judgments and why normalisation makes scores comparable across queries. Numbers from Appendix A.

Companion: recall@k (#12) asks "did you find them?"; nDCG asks "did you *order* them well?" This is the retrieval-flavoured treatment; the evaluation-flavoured one is #31.

1 Why precision/recall are not enough

Recall@k treats every relevant result as equally good and ignores order: a perfect result at rank 1 and at rank 10 count the same. But users read top-down, and some results are *more* relevant than others. We need a metric that (a) rewards higher relevance, and (b) rewards putting it *earlier*. That is nDCG.

> **Intuition**
>
> Two ideas stacked. **Gain:** let a human grade each result — e.g. 3 = perfect, 2 = good, 1 = marginal, 0 = irrelevant — so the metric knows a great hit beats a so-so one. **Discount:** divide each result's gain by a growing function of its rank, so a hit at position 5 is worth less than the same hit at position 1. Sum these discounted gains, then divide by the best score achievable on this query so the result lands in $[0, 1]$ and can be averaged across queries.

2 The formula

For a ranked list with relevance grades rel_i at positions $i = 1, 2, \ldots$:

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)}, \qquad \boxed{\text{nDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}}$$

where IDCG@k is the DCG of the *ideal* ranking — the same grades sorted best-first. The discount $\log_2(i+1)$ is 1 at rank 1, 1.585 at rank 2, 2 at rank 3 …— shrinking each lower position's contribution, gently (a log, not a cliff).

3 Worked by hand

A retriever returns six results with graded relevances $\text{rel} = (3, 2, 0, 3, 1, 2)$.

Step 1 — DCG: discount each grade by its position.

rank i	rel_i	$\log_2(i+1)$	$\text{rel}_i/\log_2(i+1)$
1	3	1.0000	3.0000
2	2	1.5850	1.2619
3	0	2.0000	0.0000
4	3	2.3219	1.2920
5	1	2.5850	0.3869
6	2	2.8074	0.7124

$$\text{DCG} = 3.0000 + 1.2619 + 0 + 1.2920 + 0.3869 + 0.7124 = 6.6532.$$

Step 2 — IDCG: the best this query could do. Sort the *same* grades best-first: $(3, 3, 2, 2, 1, 0)$, and discount:

$$\text{IDCG} = \frac{3}{1} + \frac{3}{1.585} + \frac{2}{2} + \frac{2}{2.3219} + \frac{1}{2.585} + \frac{0}{2.807} = 3 + 1.8928 + 1 + 0.8614 + 0.3869 + 0 = 7.1410.$$

Step 3 — normalise.

$$\text{nDCG} = \frac{6.6532}{7.1410} = \boxed{0.9317}.$$

A strong 0.93: the list is close to ideal, dinged mainly because a perfectly-relevant result (grade 3) sits at rank 4 instead of rank 2, and an irrelevant one (grade 0) wastes rank 3.

Mini-example · why normalise at all?

DCG alone is 6.65 — but is that good? You cannot tell without knowing the best possible. A query with many highly-relevant documents has a large IDCG; a query with one marginal document has a tiny one. Dividing by IDCG puts both on the same $[0, 1]$ scale, so nDCG of different queries can be *averaged* into a single system score. Raw DCG cannot: it would let easy, relevance-rich queries dominate the mean.

4 What nDCG captures that simpler metrics miss

- **Graded relevance:** it distinguishes "perfect" from "acceptable," which binary precision/recall cannot. Essential when results have shades of quality.
- **Position sensitivity:** moving a great result from rank 4 to rank 1 raises nDCG; recall@k would not move at all.
- **Cross-query comparability:** the $[0, 1]$ normalisation lets you average fairly over an eval set — the basis of leaderboards like BEIR and MTEB.

The cost is that you need *graded* human judgments, which are more expensive than binary relevant/not. When you only have binary labels and care about the first hit, MRR (#29) is the cheaper cousin.

Equation cheat-sheet

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)}, \qquad \text{nDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k} \in [0, 1]$$

$$\text{IDCG} = \text{DCG of grades sorted best-first;} \qquad \text{nDCG} = 1 \iff \text{already ideal order}$$

Appendix A · Reference implementation

```
import numpy as np
def dcg(rels):
    return sum(g / np.log2(i + 2) for i, g in enumerate(rels))
def ndcg(rels):
    return dcg(rels) / dcg(sorted(rels, reverse=True))

rels = [3, 2, 0, 3, 1, 2]
round(dcg(rels), 4)                       # 6.6532
round(dcg(sorted(rels, reverse=True)), 4) # 7.1410  (IDCG)
round(ndcg(rels), 4)                      # 0.9317
```

Internal learning material. Numbers reproducible from the appendix code. v1.0